

BACKEND INTEGRATION REFERENCE

Smart Shopping Mall Admin Dashboard API Guide

Endpoints, payloads, response contracts, workflow rules, and implementation notes for the admin dashboard developer.

API prefix: `/api/v1`

Development base URL: `http://10.10.26.198:5000/api/v1`

Authentication: Bearer JWT for protected endpoints

Roles: `ADMIN`, `SUPER_ADMIN`

Generated: 3 August 2026

Contents

1. Quick start and shared conventions
2. Authentication and token refresh
3. Admin profile
4. User management
5. Product management
6. Order management and workflow
7. Issue management
8. Health checks and error handling
9. Endpoint checklist

Scope. This document describes the backend as currently implemented. Routes intended only for the customer application—wishlist, customer OTP login, checkout, and seller Stripe onboarding—are not the primary focus.

1. Quick start and shared conventions

Base URL

```
const baseUrl = 'http://10.10.26.198:5000/api/v1';
```

Use the deployment URL in staging/production. Do not append a trailing slash. All endpoint paths below are relative to `/api/v1`.

Authentication header

After admin login, send the access token on every protected request:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Never expose `SUPER_ADMIN_PASSWORD`, JWT secrets, Stripe keys, or refresh tokens in frontend source code. Store tokens using the dashboard's secure session strategy.

Standard success envelope

```
{
  "success": true,
  "message": "Human-readable result",
  "pagination": { "page": 1, "limit": 10, "totalPage": 3, "total": 26 },
  "data": []
}
```

`pagination` appears only on paginated endpoints. `data` may be an object, array, or `null`.

Standard error envelope

```
{
  "success": false,
  "message": "Validation Error",
  "errorMessages": [
    { "path": "email", "message": "Invalid email" }
  ]
}
```

Status	Meaning	Dashboard behavior
400	Invalid body or invalid state transition	Show field/state error; do not retry automatically.
401	Missing, invalid, or expired access token	Try token refresh once, then redirect to admin login.
403	Authenticated but insufficient role	Show “not authorized”; hide the action in the UI.
404	Resource does not exist	Show not found and refresh the list.
409	Conflict, often payment/payout readiness	Show the server message and refresh current data.

Status	Meaning	Dashboard behavior
429	Rate limit exceeded	Disable submission temporarily.
500	Server or external provider failure	Show a generic retry message and retain form state.

Pagination, filtering, sorting, and field selection

List endpoints backed by the common query builder support these query parameters:

Parameter	Example	Meaning
page	page=2	1-based page; default 1.
limit	limit=20	Items per page; default 10.
sort	sort=-createdAt	Mongo-style sort; prefix descending fields with -.
searchTerm	searchTerm=phone	Supported by users and products against their searchable fields.
fields	fields=name,email,role	Comma-separated response projection.
Model fields	status=secured	Exact-match filters on endpoints that call <code>filter()</code> .

2. Authentication

POST /auth/admin/login

Public Password login for ADMIN and SUPER_ADMIN only.

Body

```
{
  "email": "admin@example.com",
  "password": "your-admin-password"
}
```

200 response

```
{
  "success": true,
  "message": "Admin logged in successfully.",
  "data": {
    "accessToken": "eyJhbGciOi...",
    "refreshToken": "eyJhbGciOi..."
  }
}
```

Common failures: account missing/unverified/banned, non-admin account, or incorrect password.

file:///C:/Development/Smart%20Shopping%20Mall/docs/admin-dashboard-api-integration-guide.html

4/18

POST /auth/refresh-token

Public with refresh token Generates a new access token. Rotate the in-memory/current access token after success.

Body

```
{ "refreshToken": "eyJhbGciOi..." }
```

200 response

```
{
  "success": true,
  "message": "Access token generated successfully",
  "data": { "accessToken": "eyJhbGciOi..." }
}
```

Recommended frontend token flow

1. Log in through /auth/admin/login.
2. Attach **Authorization: Bearer ...** to protected requests.
3. On the first 401, call /auth/refresh-token once and replay the original request.
4. If refresh fails, clear local authentication state and redirect to admin login.
5. There is currently no backend logout/token-revocation endpoint; logout is client-side token removal.

3. Admin profile

GET /user/profile

ADMIN / SUPER_ADMIN Returns the authenticated administrator's profile.

200 response

```
{
  "success": true,
  "message": "Profile data retrieved successfully",
  "data": {
    "_id": "66aa11111111111111111111",
    "name": "Administrator",
    "role": "ADMIN",
    "email": "admin@example.com",
    "contact": "+8801700000000",
    "location": "Dhaka",
    "image": "https://cdn.example.com/profile.jpg",
    "status": "active",
    "verified": true,
    "createdAt": "2026-08-01T08:00:00.000Z",
    "updatedAt": "2026-08-03T08:00:00.000Z"
  }
}
```

PATCH /user/profile

ADMIN / SUPER_ADMIN Updates the current profile. Use **multipart/form-data** when uploading an image.

Multipart fields

Field	Type	Required	Details
data	JSON string	No	Example: {"name": "Admin Name", "email": "admin@example.com"}
image	File	No	New profile image.

200 response

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": {
    "_id": "66aa11111111111111111111",
    "name": "Admin Name",
    "role": "ADMIN",
    "email": "admin@example.com",
    "image": "https://cdn.example.com/new-profile.jpg",
    "status": "active",
    "verified": true
  }
}
```

DELETE /user/profile

ADMIN / SUPER_ADMIN Permanently deletes the authenticated account.

200 response

```
{ "success": true, "message": "User deleted successfully" }
```

This operation is destructive. Require explicit confirmation in the dashboard.

4. User management

GET /user

ADMIN / SUPER_ADMIN Returns all users with search, filter, sorting, pagination, and field selection.

Example request

```
GET /user?page=1&limit=20&searchTerm=rahman&role=USER&status=active&sort=-createdAt
```

200 response

```
{
  "success": true,
  "message": "Users retrieved successfully",
  "pagination": { "total": 42, "limit": 20, "page": 1, "totalPage": 3 },
  "data": [
    {
      "_id": "66bb22222222222222222222",
      "name": "Amina Rahman",
      "role": "USER",
      "email": "amina@example.com",
      "contact": "+8801711111111",
      "location": "Dhaka",
      "image": "https://cdn.example.com/profile.png",
      "status": "active",
      "verified": true,
      "createdAt": "2026-08-02T10:00:00.000Z",
      "updatedAt": "2026-08-02T10:05:00.000Z"
    }
  ]
}
```

The unified customer authentication flow creates/activates users via email + OTP. There is no public manual user-registration endpoint.

5. Product management

GET /products

Public Lists products. Without a status parameter, the backend defaults to `status=available`. Admin views should explicitly request the desired status.

Examples

```
GET /products?page=1&limit=20&status=available&sort=-createdAt
GET /products?page=1&limit=20&status=secured
GET /products?searchTerm=iPhone&fields=name,brand,price,status,images,seller
```

200 response

```
{
  "success": true,
  "message": "Products retrieved successfully",
  "pagination": { "total": 12, "limit": 20, "page": 1, "totalPage": 1 },
  "data": [
    {
      "_id": "66cc33333333333333333333",
      "orderId": 1042,
      "name": "iPhone 15 Pro",
      "brand": "Apple",
      "description": "Excellent condition",
      "price": 3200,
      "condition": "used",
      "images": ["https://cdn.example.com/product-1.jpg"],
      "proofOfPurchase": "https://cdn.example.com/proof.pdf",
      "status": "available",
      "seller": {
        "_id": "66bb22222222222222222222",
        "name": "Amina Rahman",
        "location": "Dhaka",
        "contact": "+8801711111111"
      },
      "createdAt": "2026-08-02T11:00:00.000Z",
      "updatedAt": "2026-08-02T11:00:00.000Z"
    }
  ]
}
```

GET /products/:id**Public** Returns one product with seller summary.**200 response**

```
{
  "success": true,
  "message": "Product details retrieved successfully",
  "data": {
    "_id": "66cc33333333333333333333",
    "orderId": 1042,
    "name": "iPhone 15 Pro",
    "brand": "Apple",
    "description": "Excellent condition",
    "price": 3200,
    "condition": "used",
    "images": ["https://cdn.example.com/product-1.jpg"],
    "status": "available",
    "seller": { "_id": "66bb22222222222222222222", "name": "Amina Rahman", "location": "Dhaka", "contact":
"+8801711111111" }
  }
}
```

POST /products

ADMIN / SUPER_ADMIN

Creates a product attributed to the authenticated account. Must use `multipart/form-data`.

Field	Type	Required	Details
<code>data</code>	JSON string	Yes	Product fields shown below.
<code>image</code>	File(s)	Yes	Between 1 and 4 images.
<code>doc</code>	File	No	Proof-of-purchase document.

`data` **JSON value**

```
{
  "name": "iPhone 15 Pro",
  "brand": "Apple",
  "description": "Excellent condition",
  "price": 3200,
  "condition": "used"
}
```

201 response

```
{
  "success": true,
  "message": "Product created successfully",
  "data": {
    "_id": "66cc33333333333333333333",
    "orderId": 1042,
    "name": "iPhone 15 Pro",
    "brand": "Apple",
    "description": "Excellent condition",
    "price": 3200,
    "condition": "used",
    "images": ["https://cdn.example.com/product-1.jpg"],
    "proofOfPurchase": "https://cdn.example.com/proof.pdf",
    "status": "available",
    "seller": "66aa11111111111111111111"
  }
}
```

PATCH /products/:id

ADMIN / SUPER_ADMIN Partially updates a product. Send only changed fields.

Body

```
{
  "price": 2999,
  "status": "available",
  "description": "Updated description"
}
```

Allowed fields: **name**, **brand**, **description**, **price**, **condition**, and **status** (**available** | **secured** | **sold**).

200 response

```
{
  "success": true,
  "message": "Product updated successfully",
  "data": {
    "_id": "66cc33333333333333333333",
    "orderId": 1042,
    "name": "iPhone 15 Pro",
    "price": 2999,
    "status": "available",
    "description": "Updated description"
  }
}
```

DELETE /products/:id

ADMIN / SUPER_ADMIN Permanently deletes the product and its S3 images/proof document.

200 response

```
{ "success": true, "message": "Product deleted successfully", "data": null }
```

Require confirmation. This deletes both the database record and uploaded assets.

6. Order management

GET /orders/admin/all

ADMIN / SUPER_ADMIN Main admin order list, populated with product, buyer, and seller.

Example request

```
GET /orders/admin/all?page=1&limit=20&status=verification&sort=-createdAt
```

200 response

```
{
  "success": true,
  "message": "Orders retrieved successfully",
  "pagination": { "total": 8, "limit": 20, "page": 1, "totalPage": 1 },
  "data": [
    {
      "_id": "66dd44444444444444444444444444",
      "orderNumber": "CLT-12345678901",
      "product": { "_id": "66cc33333333333333333333333333", "name": "iPhone 15 Pro", "images":
[https://cdn.example.com/product-1.jpg], "status": "secured" },
      "buyer": { "_id": "66ee55555555555555555555555555", "name": "Buyer Name", "email": "buyer@example.com",
"contact": "+8801722222222", "location": "Dhaka" },
      "seller": { "_id": "66bb22222222222222222222222222", "name": "Seller Name", "email": "seller@example.com",
"contact": "+88017333333333", "location": "Dhaka" },
      "price": 3200,
      "platformFee": 384,
      "sellerPayout": 2816,
      "deliveryDetails": { "address": "House 1, Road 2", "location": "Dhaka", "phone": "+8801722222222" },
      "payment": { "provider": "stripe", "paymentIntentId": "pi_...", "status": "paid" },
      "payoutStatus": "pending",
      "status": "verification",
      "statusHistory": [{ "status": "verification", "note": "Item received", "changedAt": "2026-08-
03T08:00:00.000Z" }],
      "createdAt": "2026-08-03T07:00:00.000Z",
      "updatedAt": "2026-08-03T08:00:00.000Z"
    }
  ]
}
```

GET /orders/:id

ADMIN / SUPER_ADMIN Returns one order with populated product, buyer, and seller. Response uses the same order shape shown above.

200 response

```
{
  "success": true,
  "message": "Order details retrieved successfully",
  "data": { "_id": "66dd44444444444444444444444444", "orderNumber": "CLT-12345678901", "status": "verification",
"payment": { "status": "paid" }, "payoutStatus": "pending", "product": {}, "buyer": {}, "seller": {} }
}
```

PATCH /orders/:id/status

ADMIN / SUPER_ADMIN Advances an order through the controlled workflow. Invalid transitions return 400.

Body

```
{
  "status": "collection_pending",
  "note": "Collection scheduled for tomorrow"
}
```

200 response

```
{
  "success": true,
  "message": "Order status updated successfully",
  "data": {
    "_id": "66dd44444444444444444444",
    "orderNumber": "CLT-12345678901",
    "status": "collection_pending",
    "statusHistory": [
      { "status": "secured", "changedAt": "2026-08-03T07:20:00.000Z" },
      { "status": "collection_pending", "note": "Collection scheduled for tomorrow", "changedAt": "2026-08-03T08:30:00.000Z", "changedBy": "66aa11111111111111111111" }
    ]
  }
}
```

Allowed transitions

Current status	Allowed next status	Side effect
pending_payment	secured	Normally driven by Stripe webhook.
secured	collection_pending, cancelled	Cancellation may refund/relist.
collection_pending	collected, cancelled	Cancellation may refund/relist.
collected	verification	Item enters admin verification.
verification	payout_processing, refunded	Payout triggers Stripe transfer; refund relists.
payout_processing	ready_for_delivery	Proceed after payout succeeds.
ready_for_delivery	delivered	Product becomes sold.
delivered	completed	Product remains sold.
completed, refunded, cancelled	None	Terminal state.

PATCH /orders/:id/payout

ADMIN / SUPER_ADMIN Starts/retries the seller Stripe transfer. No request body.

Requirements

- Order status must be **verification** or **payout_processing**.
- Buyer payment must be **paid**.
- Seller must have a connected Stripe account with onboarding complete and payouts enabled.

200 response

```
{
  "success": true,
  "message": "Seller payout marked as paid",
  "data": {
    "_id": "66dd44444444444444444444",
    "orderNumber": "CLT-12345678901",
    "status": "verification",
    "payoutStatus": "paid",
    "payoutTransferId": "tr_...",
    "sellerPayout": 2816
  }
}
```

Possible conflict: seller has no connected Stripe account, onboarding is incomplete, or payouts are disabled.

7. Issue management

GET /issues

(ADMIN / SUPER_ADMIN) Returns all issues newest-first with populated product, seller, buyer, and admin. This endpoint is not currently paginated.

200 response

```
{
  "success": true,
  "message": "Issues retrieved successfully",
  "data": [
    {
      "_id": "66ff66666666666666666666",
      "product": { "_id": "66cc33333333333333333333", "name": "iPhone 15 Pro", "status": "secured" },
      "buyer": { "_id": "66ee55555555555555555555", "name": "Buyer Name", "email": "buyer@example.com" },
      "seller": { "_id": "66bb22222222222222222222", "name": "Seller Name", "email": "seller@example.com" },
      "admin": { "_id": "66aa11111111111111111111", "name": "Administrator", "email": "admin@example.com" },
      "issueType": "verification_failed",
      "reason": "Serial number does not match the proof document",
      "resolved": false,
      "createdAt": "2026-08-03T09:00:00.000Z",
      "updatedAt": "2026-08-03T09:00:00.000Z"
    }
  ]
}
```

GET /issues/:id

(ADMIN / SUPER_ADMIN) Returns one fully populated issue.

200 response

```
{
  "success": true,
  "message": "Issue details retrieved successfully",
  "data": { "_id": "66ff66666666666666666666", "issueType": "verification_failed", "reason": "Serial number mismatch", "resolved": false, "product": {}, "buyer": {}, "seller": {}, "admin": {} }
}
```

POST /issues

ADMIN / SUPER_ADMIN Opens an issue for a product. Only one unresolved issue may exist per product.

Body

```
{
  "productId": "66cc33333333333333333333",
  "issueType": "verification_failed",
  "reason": "Serial number does not match the proof document"
}
```

Allowed **issueType**: **buyer_refused** or **verification_failed**.

201 response

```
{
  "success": true,
  "message": "Issue created successfully",
  "data": {
    "_id": "66ff66666666666666666666",
    "product": "66cc33333333333333333333",
    "buyer": "66ee55555555555555555555",
    "seller": "66bb22222222222222222222",
    "admin": "66aa11111111111111111111",
    "issueType": "verification_failed",
    "reason": "Serial number does not match the proof document",
    "resolved": false
  }
}
```

If the associated order was paid, creating an issue automatically refunds the buyer. If the seller was already paid, the Stripe transfer is reversed first. The seller is emailed.

PATCH /issues/:id/resolve

ADMIN / SUPER_ADMIN Resolves an open issue and applies the selected product action.

Body: delete product

```
{ "action": "delete" }
```

Body: relist product

```
{ "action": "make_available" }
```

200 response

```
{
  "success": true,
  "message": "Issue resolved successfully",
  "data": {
    "_id": "66ff66666666666666666666",
    "product": "66cc33333333333333333333",
    "issueType": "verification_failed",
    "reason": "Serial number mismatch",
    "resolved": true,
    "updatedAt": "2026-08-03T10:00:00.000Z"
  }
}
```

delete removes the product and S3 assets. **make_available** clears the buyer and relists the product. Both actions notify the seller by email.

8. Health checks and implementation notes

GET /health/live

Server process health; this path is outside /api/v1.

```
{ "status": "ok" }
```

GET /health/ready

Returns 200 when MongoDB is connected, otherwise 503; this path is outside /api/v1.

```
{ "status": "ready" }
```

Dashboard implementation recommendations

- Centralize the base URL and bearer-token injection in one HTTP client.
- Use one refresh queue so simultaneous 401 responses do not trigger multiple refresh calls.

- Drive order action buttons from the allowed-transition table, then still handle server-side 400/409 responses.
- After status, payout, issue, product update, or deletion mutations, invalidate/refetch the affected list and detail queries.
- Show destructive confirmation dialogs for account deletion, product deletion, and issue resolution with `delete`.
- Display monetary values using the product/order currency configured for the project; backend numeric values are major currency units.
- Treat server response messages as user-facing context, but do not depend on exact text for application logic.

9. Admin endpoint checklist

Method	Endpoint	Auth	Body	Purpose
POST	<code>/auth/admin/login</code>	Public	Email, password	Admin authentication
POST	<code>/auth/refresh-token</code>	Refresh token in body	Refresh token	Renew access token
GET	<code>/user/profile</code>	Admin	—	Current admin profile
PATCH	<code>/user/profile</code>	Admin	Multipart data/image	Update current profile
DELETE	<code>/user/profile</code>	Admin	—	Delete current account
GET	<code>/user</code>	Admin	—	Browse users
GET	<code>/products</code>	Public	—	Browse/filter products
GET	<code>/products/:id</code>	Public	—	Product details
POST	<code>/products</code>	Admin	Multipart product data/files	Create product
PATCH	<code>/products/:id</code>	Admin	Changed fields	Edit product
DELETE	<code>/products/:id</code>	Admin	—	Delete product/assets
GET	<code>/orders/admin/all</code>	Admin	—	Browse/filter all orders
GET	<code>/orders/:id</code>	Admin	—	Order details
PATCH	<code>/orders/:id/status</code>	Admin	Status, optional note	Advance workflow
PATCH	<code>/orders/:id/payout</code>	Admin	—	Start/retry seller payout
GET	<code>/issues</code>	Admin	—	List issues
GET	<code>/issues/:id</code>	Admin	—	Issue details
POST	<code>/issues</code>	Admin	Product, type, reason	Create issue/refund if needed
PATCH	<code>/issues/:id/resolve</code>	Admin	Resolution action	Resolve issue

Smart Shopping Mall Admin Dashboard API Integration Guide · Generated from the backend route, validation, controller, service, and model implementation on 3 August 2026.